Republic of the Philippines
CAMARINES SUR POLYTECHNIC COLLEGES
Nabua, Camarines Sur

COLLEGE *of* COMPUTER STUDIES

Computer Vision

Camera Based Tape Path Detection and Scene Recognition

Midterm Project Report

Assigned specialization Lane Detection

Submitted by

Gabriel S. Concepcion

Ma. Francheska R. Recierdo

John Kenneth M. Bañares

Marc Dexter S. Sael

Francisco III Verga

Justine Aaron Sarcauga

BSCS-4B

A vision prototype for a future autonomous 4WD robot

Current implementation processes recorded images and videos. Robot motor control and autonomous navigation are planned for the final project.

Introduction

The course project requires a camera-based 4WD robot to progress from visual recognition at midterm to camera-influenced decisions and physical actions at finals. The midterm test asks whether the system can capture, process, detect, recognize, or interpret relevant visual information. For the assigned Lane Detection specialization, the immediate task is to detect and track a designated lane or path.

Our project represents the path with one black tape guide line rather than two road-lane boundaries. A crossing uses an additional tape branch. This controlled setup allows the group to study perception and explain each processing stage before adding motors and control. Tile seams and reflections remain relevant distractions, so recognizing a scene label alone is insufficient: the system also shows the selected tape pixels and estimated centerline.

Project objectives

The general objective is to develop and demonstrate a working vision system that detects the designated tape path and interprets its visible geometry.

- Prepare and review an original dataset with separate development, validation, and test partitions.
- Implement preprocessing, tape segmentation, centerline estimation, and recognition of the five defined scenes.
- Process recorded video and maintain a smoothed near-path offset across frames.
- Evaluate scene recognition, preserve error evidence, and explain the limits of the measurements.
- Provide a reproducible image and video demonstration that can process a newly selected input.

Scope and project status

The implemented system accepts still images, recorded video, and an optional connected camera input. The verified inputs are the group images and a development recording. The designated environment is an indoor light tiled floor with black tape viewed from a low, forward-and-downward camera position. Supported scene categories are Straight, Left curve, Right curve, Intersection, and No visible path.

The midterm demonstration runs the vision pipeline on the laptop. A robot chassis and Arduino are available to the group but are not integrated into the current vision program. No motor commands, physical steering, route selection, or autonomous mission results are claimed.

Technical background and related work

Classical computer vision

Image processing and geometric model fitting provide a practical basis for extracting structure from a controlled visual scene. Szeliski presents classical and learned approaches as part of the wider computer vision problem-solving process. In this project, strong tape-to-floor contrast allows explicit processing rules to be inspected through intermediate images. The main tradeoff is sensitivity to changes in lighting, surface appearance, and camera geometry.

Learned lane detection

Pan and colleagues introduced Spatial CNN to capture spatial relationships within feature maps for traffic-scene understanding, including lane detection [2]. That work addresses complex road scenes and motivates a possible learned approach if our future operating conditions exceed the classical baseline. Spatial CNN was not implemented or evaluated here, and its benchmark results are not directly comparable with our five-category indoor scene-classification results.

Segmentation and scene recognition

Segmentation identifies tape pixels, whereas scene recognition assigns a category to the whole image. The path mask has two conceptual pixel labels: designated tape and background. A Left curve image therefore has a scene label and may also have a separate tape mask. At an intersection, the mask includes visible tape branches. No visible path corresponds to an all-background ground-truth mask. The current dataset provides scene labels; independent geometric annotations remain a further evaluation task.

The selected midterm approach is classical OpenCV processing with hand-developed geometry rules. Development images support algorithm construction and parameter choice, rather than optimization of neural-network weights.

Dataset preparation

Source and recording procedure

The group collected its own indoor tape-path recordings using a phone. Clips were approximately 30 seconds long. Development recordings varied the camera position around the path and used stated heights of approximately 22 cm, 25 cm, and 28 cm. Additional recordings were assigned to validation and testing with separately arranged setups. All selected supplied images are 1920 by 1080 pixels. No public road dataset was used for implementation or evaluation.

Scene definitions

Category	Visible evidence used for labeling
Straight	Tape continues approximately straight; its left or right image position does not change the category.
Left curve	The designated tape visibly bends toward camera-left ahead.
Right curve	The designated tape visibly bends toward camera-right ahead.
Intersection	A designated tape crossing or branch is visible; tile-seam crossings do not count.
No visible path	No designated tape is visible. Floor seams and shadows can still be present.

Labels refer to the visible frame, not the entire recording. An approach to a curve can still be Straight when the bend is outside the image. Ambiguous transition frames should be reviewed rather than assigned a category solely from the recording folder.

Dataset composition

Category	Development	Validation	Test	Total
Straight	120	20	20	160
Left curve	120	20	20	160
Right curve	120	20	20	160
Intersection	120	20	20	160
No visible path	120	20	20	160
Total	600	100	100	800

The split is 75% development, 12.5% validation, and 12.5% testing. It comprises 12, 2, and 2 clips per category respectively, with ten selected images per clip.

Frame selection and quality checks

The extraction script saved candidate frames at approximately two-second intervals. The group reviewed candidates, selected ten useful images per recording, and supplied a final scene label for each selected image. The review table preserved source-video identity, split, extraction time, and frame index. Whole recordings remained in one split. Separate recording setups were used as a collection precaution; file checks alone cannot establish scene independence.



Figure 1. Representative development images from the group dataset. These are actual collected scenes, not synthetic training examples.

Automated checks found 800 readable images in total. The training, validation, and test archives matched their supplied manifest entries. No byte-identical duplicates were found within the checked partitions or across the checked test-to-development and test-to-validation comparisons. Representative frames from each recording and development errors were visually inspected. This review does not certify every label or rule out perceptually similar frames.

Separate geometric annotations

The scene label table is separate from geometric ground truth. Predicted tape masks and centerlines are saved as algorithm output, but have not been treated as correct annotations. To measure localization, a reviewer must independently outline the designated tape or mark center points at defined image heights. Crossing branches should be included, and no-path images should receive empty tape masks. The current report therefore does not present mask intersection over union or centerline-error scores.

Computer vision methodology

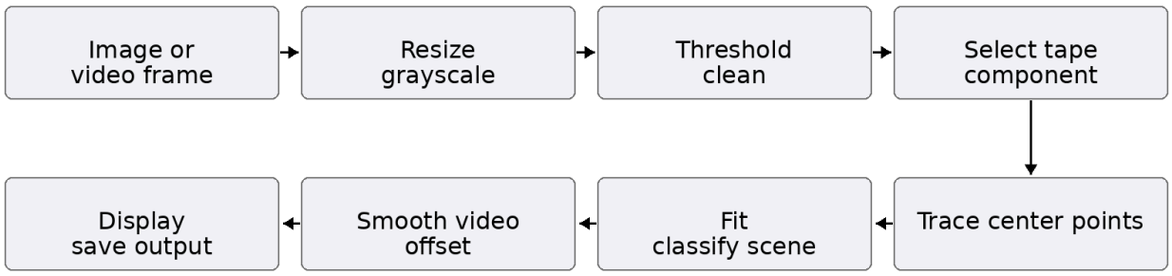


Figure 2. Implemented image and video processing workflow. Temporal smoothing applies only when processing a sequence.

Preprocessing and tape candidates

Each image is resized to 640 pixels in width while retaining aspect ratio; the supplied images become 640 by 360. The program converts the image to grayscale and applies a 3 by 3 Gaussian blur. A brightness-derived global threshold identifies dark pixels. OpenCV provides inverse binary thresholding for this purpose [3].

$$T = clip(0.60 \times median(Gfloor),\ 45,\ 110)$$

Here, Gfloor is the blurred grayscale area between approximately 35% and 95% of image height, and clip limits the threshold to the stated intensity range. Pixels at or below T become white candidates. The top 2.5% of the image and approximately 1% at each side are excluded. This is a global per-frame brightness rule, not OpenCV adaptiveThreshold.

A 3 by 3 closing operation fills small candidate gaps, followed by a 2 by 2 opening operation that removes small structures. These operations use the morphological mechanisms described in the OpenCV documentation [4]. The chosen sizes preserve more thin distant tape than the first development variant, but cannot recover arbitrary occlusions or strong glare.

Connected component selection

Connected components group adjacent candidate pixels and expose region size and position statistics [5]. A candidate must occupy at least 0.1% of the processing image, span at least 42% of its height, and reach at least 82% down the image. A geometric score favors tall supported regions with adequate area and a modest center preference. The selected component becomes the predicted tape mask. If none qualifies, the system reports no supported path.

Centerline estimation and scene rules

The program scans horizontal bands from the near field toward the far field. It follows plausible narrow tape runs and uses their centers as image-coordinate path points. Broad crossing rows are skipped when estimating the backbone. At least 12 supported center points spanning 35% of image height are required. A quadratic is fitted using NumPy polynomial least squares [6], with isolated fit outliers rejected before refitting.

$$xnorm(t) = a\,t^2 + b\,t + c$$

The horizontal coordinate is normalized by processing width, and t increases from the image bottom toward the top. The fitted coefficient a provides a signed curvature cue in image coordinates. It is not curvature in physical distance units. A slanted straight path can have a nonzero linear term b without being a curve.

Rule	Scene output
No qualifying component or insufficient centerline support	No visible path with a diagnostic status
a is below -0.065	Left curve
a is above +0.065	Right curve
a falls between these thresholds	Straight
Supported crossbar detected on the selected tape	Intersection overrides the curve or straight label

Crossbar detection uses probabilistic Hough line segments [7]. A segment must be long and approximately horizontal, cross the fitted backbone, and have supporting backbone points above and below it. The minimum segment length is 38% of image width. This support requirement helps reject distractions but explains why a far crossing near the top edge can initially be missed.

Temporal offset tracking

$$offset = (xnear - W/2)/(W/2)$$

Negative offset places the near-path center to camera-left; positive offset places it to camera-right. The video mode smooths this offset with a current-frame weight of 0.35. It resets the state on detection loss or a large offset jump. This is basic temporal smoothing, not optical flow, metric localization, or route planning. At an intersection the program displays crossing evidence without deciding a turn.

All coordinates refer to the resized processing frame. Scene outputs are deterministic rules, not calibrated probability estimates. A No visible path prediction may also represent a missed detection and should be interpreted alongside the diagnostic status and image.

System implementation

Component	Implemented use
Phone recordings	Original image and video source

Component	Implemented use
Group laptop	Confirmed local execution of image and video demonstrations
Python OpenCV and NumPy	Image processing, geometry, sequence processing, and evaluation
lane_core.py	Tape segmentation, centerline points, classification rules, and offset smoothing
lane_demo.py	Image, video, optional camera, and evaluation commands
config.json	Fixed algorithm parameters saved with evaluation outputs

The automated evaluation record identifies Python 3.12.14, OpenCV 4.14.0, and a Linux execution environment. Those versions describe the recorded evaluation environment, not an independently measured laptop benchmark. The group separately confirmed successful local image and video operation. No model training, dedicated robot camera integration, or motor-control experiment was performed for these results.

Evaluation design

Development images were used to construct the rules and select an initial configuration. That same configuration classified the validation images without further changes. The existing code and configuration were recorded before testing and verified unchanged afterward. Test labels were used for comparison only. Input filenames and category folders are not used by the detector as predictive features; the evaluator reads folders to obtain reference scene labels.

For each category, true positives count correct predictions of that category, false positives count other categories predicted as it, and false negatives count its images predicted otherwise. The report uses the following definitions:

$$Accuracy = correct\ scene\ predictions / evaluated\ images$$

$$Precision = TP / (TP + FP) \quad Recall = TP / (TP + FN)$$

$$F1 = 2 \times Precision \times Recall / (Precision + Recall)$$

Macro F1 is the unweighted mean of the five category F1-scores. Confusion matrices place reference labels in rows and predicted labels in columns. Unreadable images are logged separately; all supplied images were readable. Saved configuration files, predictions, and code hashes support reproduction. Mask accuracy and physical navigation performance require different reference data and are outside the measurements reported here.

Results and discussion

Scene recognition across partitions

Partition	Correct	Images	Accuracy	Macro F1
Development	593	600	98.83%	0.9884
Validation	100	100	100.00%	1.0000
Test	100	100	100.00%	1.0000

The development score is a measurement on data used to design the algorithm, so it should not be presented as generalization performance. Validation and test results are separate. A perfect score on these small controlled subsets does not imply perfect detection in other conditions. Their images are correlated frames drawn from ten recordings per subset, rather than 100 independent environments.

Test results by category

Category	Support	Precision	Recall	F1
Straight	20	1.000	1.000	1.000
Left curve	20	1.000	1.000	1.000
Right curve	20	1.000	1.000	1.000
Intersection	20	1.000	1.000	1.000
No visible path	20	1.000	1.000	1.000

Each test category had 20 supplied images, and all reference scene labels were matched. The test confusion matrix contains 20 on every diagonal entry and zero off the diagonal. No test labels, input images, or parameters were modified to obtain these scores.

Development errors

Seven development images were misclassified: five Intersection images as Straight, one Right curve as Straight, and one Straight image as Right curve. Error panels were retained. The distant-crossing errors are consistent with the requirement for backbone support above and below a crossbar. The straight-versus-curve errors show sensitivity to weak curvature and uneven tape appearance. These examples were not removed merely to raise the score.

The lower development score is compatible with the validation and test outcomes: the particular frames and difficult viewpoints differ between partitions. These results do not establish that every validation or test setup is more difficult than the development set, and no statistical independence is assumed.

Test confusion matrix

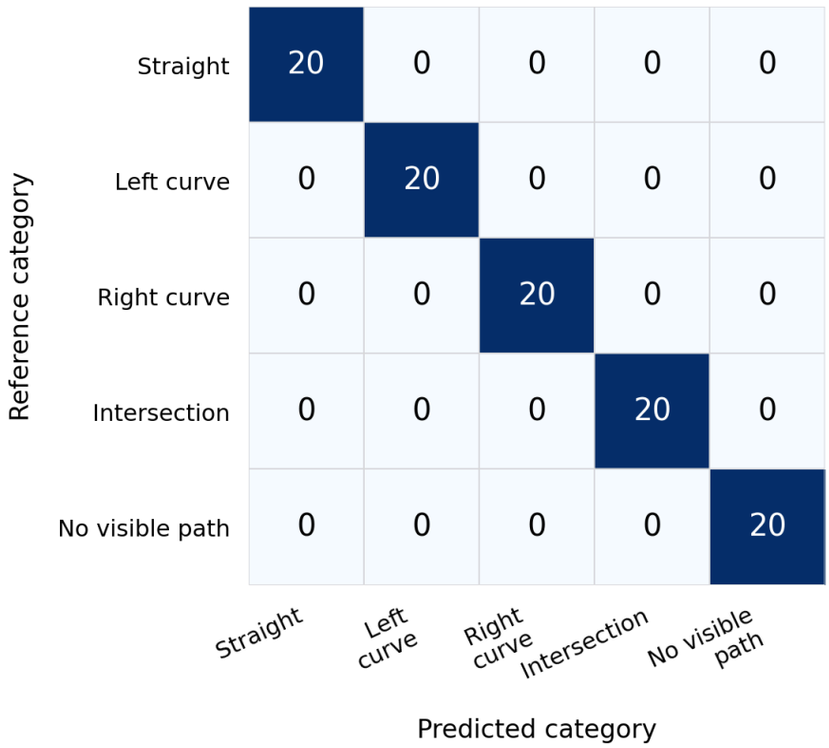


Figure 3. Scene-classification confusion matrix for the supplied 100-image test set. Rows are reference labels; columns are predictions.

All five categories were recognized correctly in the supplied image test. The matrix measures the category assigned to each frame; it does not measure the overlap of the predicted tape mask with a manual mask. Likewise, correct recognition of Intersection does not determine whether the robot should turn left, turn right, or continue forward.

The absence of byte-identical cross-split images is a useful file check, but similar backgrounds and video-frame correlation remain. Generalization should be examined later with additional independently arranged tracks, lighting conditions, and camera positions. Any future tuning based on these test observations should be followed by evaluation on a newly reserved set.

Qualitative path output

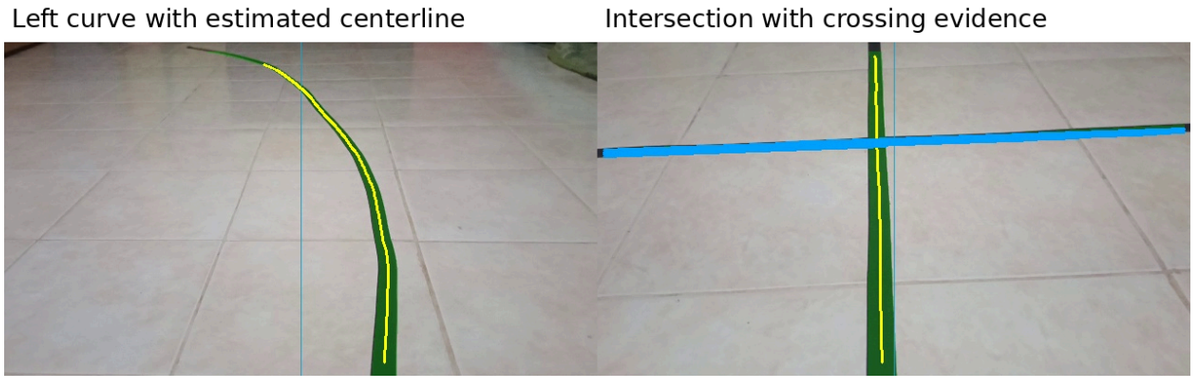


Figure 4. Actual prototype overlays on development examples. Green marks selected tape, yellow marks the estimated backbone, and blue marks detected crossing evidence.

The left-curve example shows the estimated centerline following the visible tape bend while retaining a separate camera-center reference. The intersection example shows the main backbone and the crossing tape. These visual outputs make the processing result inspectable during the demonstration and support the assigned path-detection task beyond a text category label.

The program also saves a grayscale image, dark-pixel candidates, a selected tape mask, and numerical result information. A selected component can differ from the true tape region if shadows, glare, or connected floor markings interfere. The displayed centerline is therefore a prediction. No statement of centimeter accuracy or pixel-level correctness follows from these examples.

Runtime interpretation

Recorded evaluation	Mean detector time
Development image set	6.14 ms per image
Validation image set	6.05 ms per image
Test image set	5.76 ms per image
Development video	9.75 ms per frame

These times were logged in the evaluation environment and cover the detector, with offset smoothing included in the video measurement. They exclude decoding, rendering, output writing, and display. They are not end-to-end latency or measured laptop throughput. Successful local execution has been confirmed, but a hardware-specific laptop speed benchmark has not been recorded.

Development video review

The supplied Intersection development recording has a duration of approximately 30.7 seconds and 920 decoded frames. The unchanged detector processed all frames and generated a pipeline video and prediction log. The camera approaches the crossing; a clear post-crossing departure and disappearance of the path are not covered by this clip.

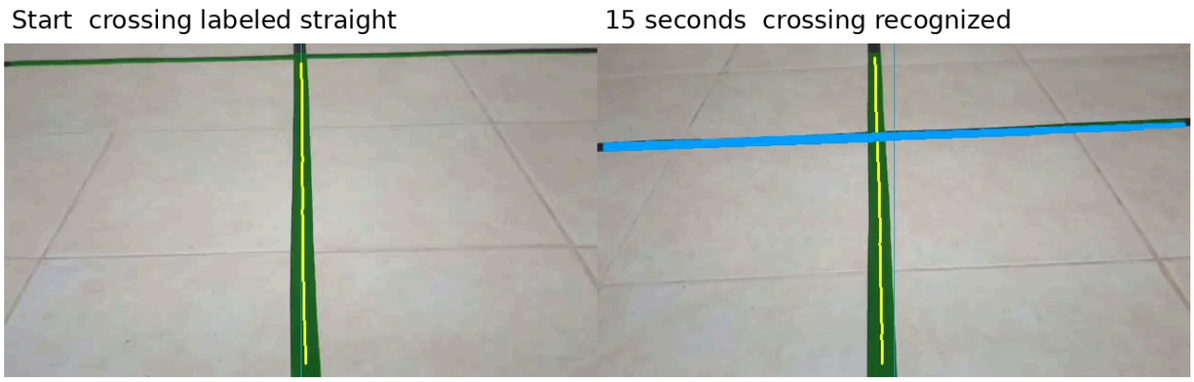


Figure 5. Early distant-crossing failure and later crossing recognition from the same development video. The unchanged detector initially reports Straight.

Video observation	Recorded result
Frames processed	920
Frames with a supported path reported	920
Frames labeled Intersection	863
Frames labeled Straight	57
Stable Intersection label through the remaining clip	From approximately 2.17 seconds

The first distant crossing is labeled Straight, followed by brief label switching near 1.8 to 2.17 seconds. The remaining 855 frames are consistently labeled Intersection. The crossing-support rule explains this failure near the upper image boundary. The observation should be shown during the demo rather than hidden by starting playback after the difficult section.

These counts are diagnostic outputs, not a video-accuracy score. There are no independent per-frame reference labels or geometric annotations. A supported path in every frame does not establish correct tape geometry in every frame. Timestamps in the saved prediction log use nominal frame rate and are approximate for variable-rate footage.

Limitations

The dataset covers one controlled indoor tape-path task with limited physical setups and many related frames. The tests do not establish reliability on outdoor road lanes, heavy occlusion, strong shadows, damaged tape, or large changes in camera orientation. Exact-duplicate checks cannot identify every visually similar recording or establish the independence of collection sessions.

The image-geometry rules assume that the designated path has sufficient contrast and extends toward the lower image. Thin distant branches may lose support; shiny tape can create bright gaps; weak bends can be confused with straight paths. The five scene labels summarize visible geometry, while intersections may require a richer graph representation for navigation. No calibrated confidence score is produced.

Manual masks and centerline references are not available, so localization accuracy remains unmeasured. Real-video loss recovery, route selection, motor response, closed-loop stability, battery operation, and autonomous completion have not been demonstrated. Temporal smoothing reduces offset variation but is not a complete tracking or control solution.

Plan for final integration

Stage	Planned work	Evidence to collect
Perception refinement	Add independent setups and manual geometric labels; address distant crossings.	Scene metrics plus mask or centerline error.
Camera and communication	Mount a suitable camera and connect vision output to the Arduino or chosen controller.	Camera capture and command logs.

Stage	Planned work	Evidence to collect
Closed-loop movement	Map path error to steering and motor commands; define path-loss behavior and an emergency stop.	Recorded controlled robot trials.
Final evaluation	Reserve fresh unseen inputs and follow the assigned final mission requirements.	Additional unseen tests and autonomous mission results.

Conclusion

The midterm project produced a functioning tape-path vision prototype, an original reviewed dataset, and reproducible evaluation outputs. The implementation accepts images and recorded video, displays tape segmentation and estimated centerlines, recognizes five scene categories, and smooths near-path offset across frames. The group confirmed that the image and video demonstrations work on its laptop.

The recorded scene-classification results are 593 correct predictions out of 600 development images, 100 out of 100 validation images, and 100 out of 100 supplied test images. These outcomes apply to the collected indoor scenes. The development video also reveals a distant-intersection failure at the start of the clip. Independent geometric accuracy and autonomous navigation remain future evaluation tasks.

The present system provides the perception foundation for the final robot project. The next engineering steps are to strengthen geometric evaluation, improve difficult-view detection, integrate a camera and controller, and measure closed-loop robot behavior on independently reserved scenarios.

Midterm requirement evidence

Required element	Available evidence
Dataset and separate partitions	800 reviewed frames with source-video and split records.
Preprocessing and CV algorithm	Grayscale, threshold, morphology, component, and geometry outputs.
Designated path detection	Predicted tape masks and centerline overlays.
Basic decision logic	Five scene-category rules and path-loss diagnostic state.
Evaluation and confusion matrix	Separate development, validation, and test prediction records.
Working system demonstration	Image/video commands confirmed working on the laptop.

During the assessment, the group will run the algorithm on selected inputs and explain the intermediate stages. Saved figures and recordings support the report; the working program remains the basis of the demonstration.

13 References

[1] Szeliski, Richard. *Computer vision: algorithms and applications*. Vol. 1. London: Springer, 2011. https://doi.org/10.1007/978-3-030-34372-9

[2] Xingang Pan, Jianping Shi, Ping Luo, Xiaogang Wang, and Xiaoou Tang. 2018. Spatial as Deep: Spatial CNN for Traffic Scene Understanding. Proceedings of the AAAI Conference on Artificial Intelligence 32, 1. https://doi.org/10.1609/aaai.v32i1.12301

[3] OpenCV. n.d. Image Thresholding. OpenCV documentation. https://docs.opencv.org/4.x/d7/d4d/tutorial_py_thresholding.html

[4] OpenCV. n.d. Morphological Transformations. OpenCV documentation. https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html

[5] OpenCV. n.d. Structural Analysis and Shape Descriptors. OpenCV documentation, connectedComponentsWithStats. https://docs.opencv.org/4.x/d3/dc0/group__imgproc__shape.html

[6] NumPy Developers. n.d. numpy.polyfit. NumPy documentation. https://numpy.org/doc/stable/reference/generated/numpy.polyfit.html

[7] OpenCV. n.d. Hough Line Transform. OpenCV documentation. https://docs.opencv.org/4.x/d6/d10/tutorial_py_houghlines.html